

Apply filters to SQL queries

Project description

In this project, I assume the role of a security professional responsible for investigating incidents and managing system updates through database analysis. The goal is to filter large volumes of data in the `employees` and `log_in_attempts` tables to identify suspicious behavior and segment users according to their department and office. I use SQL queries with logical operators (AND, OR, NOT) and pattern filters (LIKE) to extract accurate information and improve the organization's security posture.

Retrieve after hours failed login attempts

A possible incident was detected that occurred after 6:00 PM. To investigate the failed attempts during that time, I executed the following query:

```
SELECT *  
FROM log_in_attempts  
WHERE login_time > '18:00' AND success = 0;
```

How it works: I used the AND operator to combine two mandatory conditions. The first filters the records from the `login_time` column that are after '18:00'. The second searches the `success` column for the value 0 (or FALSE), which represents a failed attempt. Only records that meet both conditions simultaneously are displayed.

Retrieve login attempts on specific dates

To analyze a suspicious event that occurred between May 8th and 9th, 2022, I used the following query:

```
SELECT *  
FROM log_in_attempts  
WHERE login_date = '2022-05-08' OR login_date = '2022-05-09';
```

How it works: The OR operator allows obtaining records that match either of the two specified dates. In this case, the database returns the login attempts that occurred on both May 8th and 9th.

Retrieve login attempts outside of Mexico

Given that the malicious activity did not originate in Mexico, I filtered international logins:

```
SELECT *  
FROM log_in_attempts  
WHERE NOT country LIKE 'MEX%';
```

How it works: I used the NOT operator to exclude records and LIKE with the % wildcard to capture any variation of the country name (such as "MEX" or "MEXICO"). By combining both, the query returns all login attempts except those originating from Mexico.

Retrieve employees in Marketing

To perform security updates on the Marketing team's computers located in the East building, I used:

```
SELECT *  
FROM employees  
WHERE department = 'Marketing' AND office LIKE 'East%';
```

How it works: This query uses AND to ensure that the employee belongs to the 'Marketing' department. The LIKE 'East%' filter ensures that all offices in the East building are included (such as East-170 or East-320) by searching for any string that begins with that word.

Retrieve employees in Finance or Sales

For the deployment of security patches in the Sales and Finance departments, I executed:

```
SELECT *  
FROM employees  
WHERE department = 'Finance' OR department = 'Sales';
```

How it works: Here, the OR operator is key to grouping employees from two distinct areas into a single report. The department column is repeated in both conditions to ensure valid SQL syntax.

Retrieve all employees not in IT

To identify employees who have not yet received a technical update (excluding the IT team), I used:

```
SELECT *  
FROM employees  
WHERE NOT department = 'Information Technology';
```

How it works: The NOT operator filters and specifically excludes members of the "Information Technology" department, allowing you to obtain a list of all other departments that require the update.

Summary

Through these tasks, I demonstrated the ability to use SQL as a forensic auditing and asset management tool. The strategic use of logical operators and pattern filters allowed me to segment access incidents and group employees efficiently. This data-driven approach is essential for any cybersecurity analyst, as it allows them to transform thousands of records into actionable information to protect the company's infrastructure.